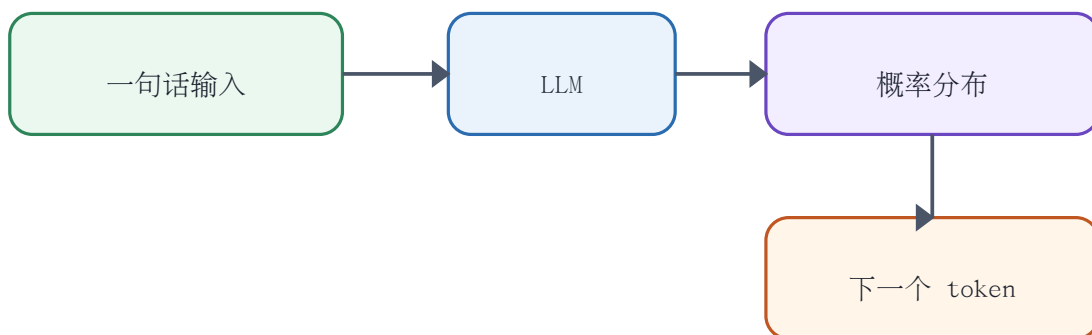


第1讲：大语言模型到底是什么？

从“下一个词预测器”建立第一套正确心智模型



适合对象：零基础学习者。默认不要求机器学习、深度学习、线性代数或 Transformer 基础。
学习路线：先形成直觉，再逐步引入术语；先知道“发生了什么”，后续课程再解释“怎么算出来”。

本讲你最终要得到什么？

核心概念 一句话目标

学完这一讲，你应该能用自己的话解释：LLM 是什么、它最基础的训练目标是什么、为什么“预测下一个 token”能够产生复杂能力、模型与 ChatGPT/Agent 为什么不是一回事，以及训练和推理有什么根本区别。

学习完成标准

看到“LLM、GPT、ChatGPT、Agent”这些词时，不再混成一个东西。

理解 LLM 生成文本时，本质上是在反复计算“下一个 token 的条件概率”。

知道“模型会说得像真的”与“模型知道事实是真的”不是同一件事。

知道“训练”是改变参数，“推理/生成”主要是在使用已经学到的参数。

能解释为什么 Agent 研究必须懂 LLM，但 Agent System 又远远不等于 LLM。

本讲目录

- 01 先从一个问题开始：语言模型到底在做什么？
- 02 什么叫“语言模型”？
- 03 什么叫“大”语言模型？
- 04 LLM 生成一句话时内部发生什么？
- 05 “预测下一个 token”为什么能学出复杂能力？
- 06 LLM 是在理解，还是在模仿？
- 07 LLM、GPT、ChatGPT、Agent 的关系
- 08 训练和推理：模型什么时候真的在“学习”？
- 09 LLM 能做什么，又天然不擅长什么？
- 10 从第一讲连接到 Agent Systems
- 11 常见误区清单
- 12 动手小实验
- 13 本讲术语表
- 14 练习与答案
- 15 一页总结与下一讲预告

1

先从一个问题开始

如果它只是“预测下一个词”，为什么看起来会思考？

假设我给你一句没写完的话：

“法国的首都是 ____。”

你几乎会立刻想到“巴黎”。这件事看起来非常简单，但如果把句子换成下面这些，难度就会逐渐上升：

“水在标准大气压下达达到 100° C 后通常会 ____。”

“如果 A 比 B 大，B 又比 C 大，那么 A 与 C 的关系是 ____。”

“下面这段 Python 为什么报错：……”

“请把这段学术论文改写成通俗中文。”

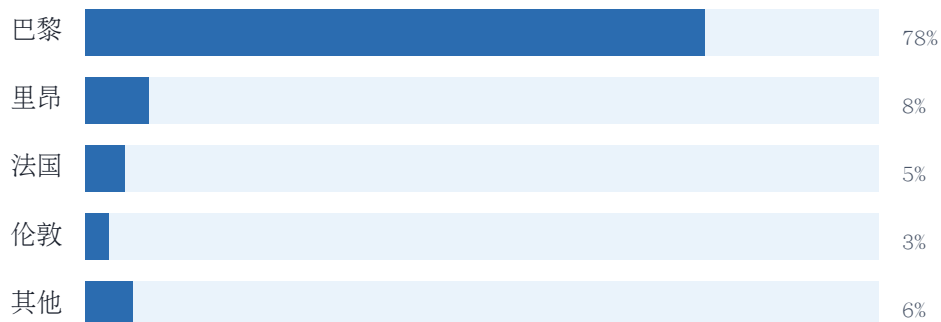
“给定前面的对话、工具返回结果和约束，下一步应该执行什么动作？”

大语言模型最原始、最核心的训练任务，可以抽象成同一个问题：给定前面的内容，预测接下来最可能出现的 token。

核心概念 第一条心智模型

先暂时不要把 LLM 想成“数据库”“搜索引擎”或“数字大脑”。第一讲最有用的抽象是：它是一个极其复杂的条件概率模型，输入上下文，输出下一个 token 的概率分布。

模型不是“想出一个词”，而是先得到一组概率



注意：上图中的百分比只是教学示意，不代表任何真实模型的实际概率。关键是理解“先有分布，再选 token”。

补充说明 为什么这里先说 token，不说“词”？

真实的大模型通常不是严格按“一个汉字”或“一个英语单词”来处理文本，而是把文本切成 token。token 的详细机制放在第2讲。现在只要把 token 临时理解成“模型处理文本时使用的小块单位”。

2

什么叫“语言模型”？

“语言”是对象，“模型”是对规律的压缩表示

2.1 “模型”这个词，别想得太神秘

在科学和工程里，“模型”通常指：用一个可计算的结构去近似现实中的某种规律。例如天气模型试图描述大气变化，房价模型试图描述价格与面积、位置等因素的关系。语言模型则试图描述：在某段文字已经出现的情况下，后面的文字有多大可能以不同方式继续。

2.2 一个最朴素的语言模型

假设我们统计了大量句子，发现“今天晚上吃”后面经常跟“饭”“火锅”“面”，那么哪怕不用神经网络，仅靠频率表也可以做一个非常粗糙的语言模型。

前文	候选后续	估计概率
今天晚上吃	饭	45%
今天晚上吃	火锅	25%
今天晚上吃	面	15%
今天晚上吃	其他	15%

现代 LLM 与这个“频率表”相比当然复杂得不可同日而语，但核心问题并没有变：根据上下文，对后续文本建立概率。区别在于，神经网络让模型能够利用非常复杂的上下文关系，而不只是查一个固定的短语表。

核心概念 底层原理直觉

传统频率模型更像“查表”；神经网络语言模型更像“把大量语言规律压进参数里，再根据当前上下文动态算概率”。后面课程会逐步解释这些参数是什么、怎样被训练出来。

2.3 “语言模型”不是“聊天机器人”的同义词

一个语言模型本身可以只负责计算概率和生成 token；聊天界面、历史记录、文件上传、联网搜索、代码运行、工具调用、安全策略等，都是更外层的系统能力。把它们混在一起，会直接影响你以后看 Agent 论文。

例子 一个现实比喻

把 LLM 想成一台很强的“语言发动机”。ChatGPT 之类的产品更像整辆汽车：除了发动机，还包括方向盘、刹车、仪表盘、导航、车身和各种控制系统。Agent 则进一步加入“目标、环境、工具、行动循环”等机制。

3

什么叫“大”语言模型？

Large 不是只指参数数量

“Large Language Model”的“大”，最直观地让人想到“参数很多”。这没有错，但如果只理解成“参数多”又太窄。今天说一个模型“很大”，通常意味着它在多个维度都采用了很大的规模：

参数规模：神经网络里可以被训练调整的数值很多。

训练数据规模：见过的文本、代码、数学、合成数据等非常庞大。

训练计算量：需要大量矩阵运算和昂贵的算力。

模型容量：能够表示和组合非常复杂的统计规律。

使用场景：不再只解决一个窄任务，而是表现出广泛的通用语言与推理能力。

常见误区 不要把“大”理解成“文件体积大所以聪明”

真正重要的是：规模扩大后，模型能够在训练过程中吸收更丰富的统计结构。参数只是承载这种结构的载体之一。后面我们还会学到 Scaling Law（尺度规律）以及数据、参数、算力之间的关系。

3.1 参数是什么？先有个直觉就够了

现在先把“参数”理解成模型内部大量可调的数字旋钮。训练的过程，就是根据预测错误不断微调这些旋钮，使模型在未来遇到相似上下文时给出更合适的概率。

例如，一个极简模型只有三个“旋钮”：

$$w_1 = 0.8 \quad w_2 = -0.2 \quad w_3 = 1.4$$

真实的大模型可能有数十亿、数百亿甚至更多参数。参数到底怎样参与神经网络计算，会在后面的神经网络基础课详细解释。

例子 把参数想成“经验被固化后的数字痕迹”

模型训练时并不是把每篇文章原样存进去，而是让训练数据不断改变参数。训练结束后，参数携带了从数据中学习到的统计规律。这个说法是理解“模型不是普通数据库”的关键。

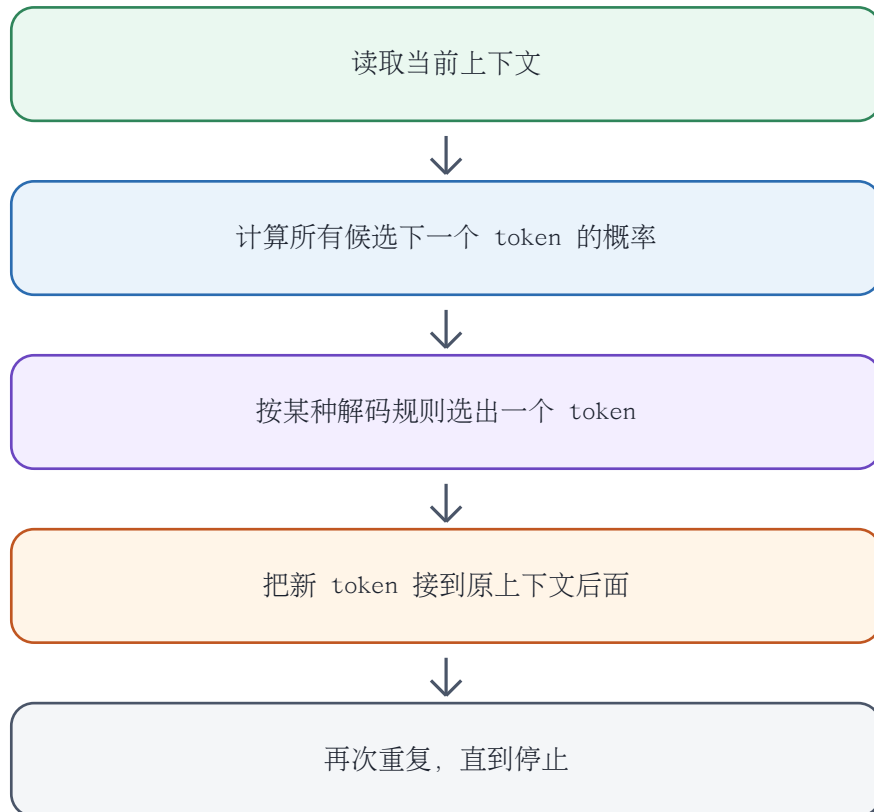
4

LLM 生成一句话时内部发生什么？

反复执行同一个循环

假设你输入：“法国的首都是”。先忽略 tokenizer、embedding、Transformer 等内部细节，只看最外层流程：

一次生成 = “预测 → 选择 → 接回去 → 再预测”



如果第一次选中了“巴”，上下文就变成“法国的首都是巴”；第二次再根据新上下文预测，可能选中“黎”；之后可能产生句号。很多模型在界面上看起来像是在“逐字打字”，本质上就是这种自回归（autoregressive）生成循环。

科研连接 为什么前面生成错了，后面可能越错越远？

因为模型后面的每一步都把自己前面已经生成的内容当作新的上下文。如果某一步产生了错误 token，错误也会进入后续条件，从而改变后面所有概率。这也是研究 Agent 可靠性时非常重要的“误差传播”直觉。

4.1 模型每一步得到的不是“唯一正确答案”

语言本身经常有多个合理续写。例如“今天的天气很____”可以是“好”“冷”“热”“闷”。因此模型输出的是一个分布，而不是天然存在一个唯一词。具体最终选哪个，除了概率本身，还受 Temperature、Top-p 等解码策略影响，这些将在后面的推理课程里详细讲。

常见误区 概率高 $\neq$ 事实真

模型计算的是“在当前上下文和训练规律下，这个 token 多合适/多常见”，而不是直接调用一个“真伪判定器”。这两种目标有重叠，但绝不是同一件事。

5

为什么“预测下一个 token”能学出复杂能力？

简单目标不等于简单内部结构

这是初学者最容易觉得“不可思议”的地方：如果训练目标只是续写，为什么模型后来会做翻译、写代码、总结论文，甚至完成一些推理？

5.1 关键：要把下一个 token 预测好，必须学会越来越多的结构

考虑下面几组句子：

例子 语法

“He ____ a student.” 要预测 is，模型必须掌握语法搭配。

例子 事实

“The capital of France is ____.” 要预测 Paris，需要吸收世界知识的统计关联。

例子 指代

“小王把书给小李，因为他已经看完了。” 要续写自然，模型要利用人物与指代关系。

例子 代码

要补全函数的下一行，模型往往需要理解变量、控制流和 API 用法。

例子 推理

某些题目中，后续答案依赖前文多个条件，模型必须形成能支持条件组合的内部表示。

也就是说，“预测下一个 token”只是训练目标；为了把这个目标做得足够好，模型内部需要形成大量中间结构。这就像一个人为了长期通过复杂考试，不能只记住答案，最终往往必须学会一定程度的规则、概念和抽象。

核心概念 重要区分：训练目标 vs 涌现出的能力

模型被直接优化的是预测损失；翻译、摘要、编程、问答等能力并不是每一种都以单独模块写死进去。它们很大程度上来自大规模训练后形成的可复用表示，再经过指令训练、偏好优化等阶段进一步变得可用。

5.2 但这并不意味着“只要会预测，就等于人类理解”

这里要保持精确：现代 LLM 的内部确实会形成丰富、可泛化的表示和算法性行为，这是大量研究观察到的；但“这是否等同于人类意义上的理解、意识或主观体验”属于更复杂的科学与哲学问题。第一讲不需要把这些概念混在一起。对工程和科研来说，更可靠的问题是：它在什么输入分布下能稳定做什么？为什么会失败？

科研连接 研究者应避免的两个极端

极端一：“它只是随机鹦鹉，所以什么都没学会。”

极端二：“它回答得像人，所以内部一定和人一样理解。”

更科学的做法是测量行为、表示、泛化边界和失败模式。

6

LLM 是在理解，还是在模仿？

先把“可观测能力”和“哲学判断”分开

对于科研入门，一个非常实用的原则是：不要急着用“理解/不理解”给模型贴总标签，而是把问题拆成可测量的能力。

宽泛说法	更可测量的问题
它理解代码吗？	能否解释 bug？能否在未见代码上修复？能否跨文件追踪状态？
它理解世界吗？	事实问答准确率如何？遇到时间变化、歧义、反事实条件会怎样？
它会推理吗？	对组合推理、长链推理、扰动输入、验证步骤是否稳定？
它会计划吗？	多步任务成功率如何？中途错误能否发现并恢复？

这套思路对你以后做 Agent Systems 科研尤其重要：不要只问“Agent 聪不聪明”，而是定义任务、环境、可观测指标和失败条件。

科研连接 一个非常重要的科研习惯

把模糊的能力判断拆成实验可测问题。例如“多 Agent 会不会更聪明”太宽；“在相同总 token 预算下，异质 Agent 能否通过降低错误相关性提升准确率”就是可以被实验检验的科学问题。

6.1 为什么它会“像懂了一样”？

因为自然语言中包含大量世界结构。一个模型若想在海量不同上下文中持续预测得好，就必须捕捉语法、语义、实体关系、代码结构、常见推理模式等统计规律。其输出因此可以呈现出高度组织化的行为。

6.2 为什么它又会突然犯很低级的错误？

因为这种能力并不是一个传统程序员手写的确定性规则系统。它依赖分布式的参数表示、当前上下文、解码过程和训练覆盖范围。只要进入未覆盖或脆弱区域，就可能出现“前面表现很强、下一秒却错得离谱”的现象。

科研连接 Agent Systems 为什么值得研究

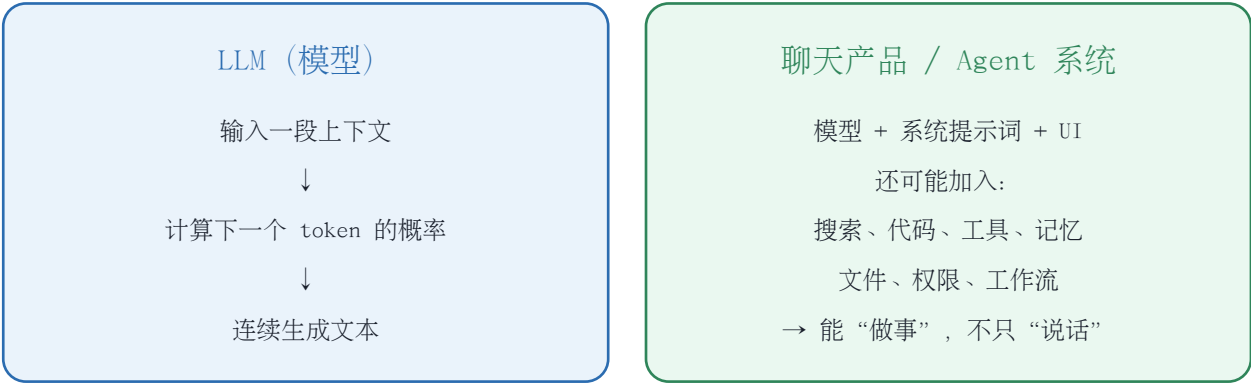
一旦 LLM 被放入长流程系统里，单次小错误可能经由记忆、工具调用、通信与环境反馈不断放大。于是研究对象从“单次回答正确率”扩展为“复杂系统怎样保持可靠”。这正是 AI Systems Science 的典型问题。



LLM、GPT、ChatGPT、Agent 的关系

四个层级，不要混淆

LLM、聊天产品、Agent：不要混为一谈



7.1 LLM：类别名

LLM（Large Language Model）是一个类别。只要属于大规模语言模型这一类，都可以叫 LLM。就像“汽车”是类别，不代表某一个具体品牌。

7.2 GPT：一种模型家族/架构路线的名称

GPT 最初来自 “Generative Pre-trained Transformer”。它强调生成式、预训练和 Transformer。GPT 是 LLM 世界中的一个重要模型家族，但 LLM 并不只有 GPT。

7.3 ChatGPT：围绕模型构建的产品/系统

一个聊天产品通常至少还包含会话管理、系统提示、安全控制、文件处理、工具、联网、代码执行、用户界面等。模型是核心组件之一，但“产品行为”不能简单等同于“裸模型行为”。

7.4 Agent：把模型置于目标-行动-反馈循环中

Agent 的典型特征不是“更会聊天”，而是能根据目标和环境状态持续选择行动，例如搜索、写文件、调用 API、运行代码、观察结果，再决定下一步。它往往还加入规划、记忆、工具权限和控制逻辑。

层级	核心问题	典型研究对象
----	------	--------

LLM	如何表示与生成语言/代码？	架构、训练、推理、对齐
聊天系统	如何把模型做成可用产品？	上下文、UI、工具、安全层
Agent	如何在环境中持续做决策与行动？	规划、工具、记忆、执行、反馈
Multi-Agent System	多个 Agent 如何相互作用？	协作、通信、分工、群体错误

科研连接 以后看论文时先问一句

作者研究的是“模型本身”，还是“模型外面的系统”？很多 Agent 论文真正的创新发生在系统层：记忆、路由、角色、通信、工具权限、验证、调度、预算分配，而不是重新训练一个基础模型。

8

训练和推理：模型什么时候真的在学习？

这是后面所有课程的基本分界线

训练（Training）与使用（Inference）是两件不同的事

训练：改变参数

海量文本
↓
预测下一个 token
↓
计算误差
↓
调整参数
↓
重复很多次

推理：参数基本不变

用户输入
↓
模型计算概率
↓
选择一个 token
↓
把新 token 接回去
↓
继续生成

8.1 训练：把经验写进参数

训练时，模型看到大量样本，做预测，计算预测与目标之间的误差，再用优化算法调整参数。这个过程会真正改变模型内部的权重。

可以把它写成一个最简循环：

数据 → 预测 → 误差 → 调参数 → 再预测

8.2 推理：使用已经训练好的参数

当你平时问模型问题时，通常发生的是

inference（推理/生成）。模型会使用已经训练好的参数，根据当前上下文不断预测 token。一次普通对话并不意味着基础模型立刻重新训练。

常见误区 为什么“我刚教了它一句话”不等于“模型参数已经学会”

对话里的新信息通常进入的是上下文，不是基础模型参数。上下文结束后，这段信息未必还在。某些产品另有记忆系统，但那也属于外层系统机制，不能和神经网络参数训练混为一谈。

8.3 训练、微调、上下文学习不是一回事

方式	是否改参数	作用
预训练	是	从大规模数据中学习通用规律
微调 / 对齐	通常是	让行为更符合特定任务或偏好
上下文学习	否	临时利用 prompt 中提供的示例和信息
外部记忆	模型参数通常不变	由系统保存并在未来重新放回上下文

9

LLM 能做什么，又天然不擅长什么？

能力来自统计学习，边界也来自统计学习

9.1 它特别擅长的事情

对语言和代码做模式识别、转换和生成。

总结、改写、翻译、解释、分类、信息抽取。

根据已有模式生成代码、测试、文档和结构化文本。

在某些任务上执行多步推理、规划和工具使用。

把自然语言当作通用接口，连接人类意图与软件工具。

9.2 它不天然保证的事情

事实真实性：生成“最合理的语言”不等于“事实已被验证”。

最新信息：参数里的知识受训练时间影响；没有外部工具时无法自动知道刚发生的事件。

确定性：生成过程和上下文变化可能导致答案波动。

长期一致性：长任务中错误会累积，目标可能漂移。

权限边界：一旦接入工具，系统必须额外设计访问控制、验证与审计。

自我验证：模型可以生成检查步骤，但“自己说自己对”不构成真正独立验证。

核心概念 幻觉不是简单的“模型坏了”

只要系统的生成目标主要还是语言概率，就可能出现“形式非常合理、内容却不真实”的输出。降低幻觉可以依靠检索、工具、训练、验证器和系统设计，但从原理上不能把语言流畅度当成事实校验。

科研连接 这正是可靠 Agent 的难点

Agent 把 LLM 的输出变成现实动作后，错误代价会提高。一个错误答案只是文字；一个错误工具调用可能修改文件、错误下单或污染长期记忆。因此 Agent 研究必须把“生成能力”与“控制、验证、恢复”一起看。

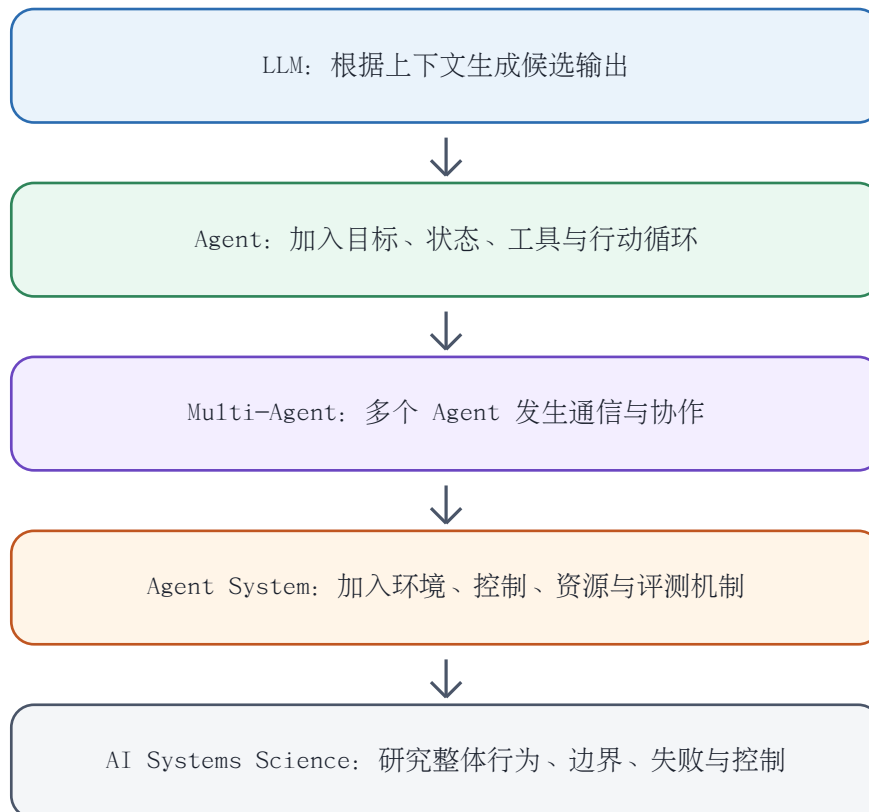
10

从第一讲连接到 Agent Systems

LLM 是智能组件，系统科学研究的是相互作用

现在可以给出一条非常重要的层级关系：

从“模型”上升到“系统”



因此，你未来做 Agent Systems，不需要先成为“训练基础模型”的专家，但必须理解 LLM 的几个底层特征：

它是概率性的，而不是传统确定性程序。

它的能力依赖上下文，且上下文有长度和注意力分配问题。

它会犯错，而且错误可能有相关性和系统性模式。

它的输出可以被当成其他 Agent 或工具的输入，因此错误会传播。

它可以通过提示词、角色、经历、记忆、工具等产生不同的行为分布。

科研连接 一个你以后会反复遇到的研究问题

如果多个 Agent 都由类似的 LLM 驱动，它们是不是会犯相似的错误？如果长期经历不同，会不会形成不同的功能性偏差，从而降低错误相关性？这类问题必须建立在“LLM 是概率模型、上下文会改变输出分布”的基础上理解。

11

常见误区清单

第一讲就把最容易走偏的地方纠正

常见误区 误区1: LLM 就是一个超大数据库

数据库强调“存储与精确检索记录”；LLM

参数更多是对数据规律的压缩表示。它可以记住部分事实，但输出不是简单数据库查询。

常见误区 误区2: 模型每次都在互联网搜索

裸 LLM 推理不等于联网搜索。只有外层系统显式调用搜索工具时，才会获取网络信息。

常见误区 误区3: 它说得越自信就越正确

语言风格和事实正确性不是同一个优化目标。流畅、完整、自信都不能当作证据。

常见误区 误区4: ChatGPT = GPT 模型

ChatGPT 是产品/系统；模型是其中的核心组件之一。系统层还可能包含工具、记忆、安全层等。

常见误区 误区5: 每次聊天都会训练模型参数

普通推理通常不改基础模型参数。当前对话信息主要通过上下文被利用。

常见误区 误区6: 预测下一个 token 很简单，所以模型内部一定很简单

训练目标可以简单，但为了把它做得好，模型内部可以形成极其复杂的表示和计算过程。

常见误区 误区7: LLM 既然会推理，就不会犯基础错误

能力是分布性的，不是一个“会/不会”的总开关。模型在某些分布上很强，在另一些扰动或长流程条件下仍可能脆弱。

12

动手小实验

不用代码，也可以亲手看到语言模型思维方式

动手实验 实验目标

亲自体验“同一上下文可以有多个候选后续”“上下文会改变概率”“生成结果会反过来影响后续生成”这三个核心现象。

实验 A：你自己就是一个“人工语言模型”

请不要看答案，先给下面每句话写出你认为最可能的 3 个后续，并给一个粗略概率。概率总和约等于 100% 即可。

1. “明天早上我准备先 ____。”
2. “法国的首都是 ____。”
3. “如果服务器磁盘已经 100% 满了，下一步最应该先 ____。”

观察：第1题开放性最大，候选很多；第2题事实约束很强；第3题需要领域知识和操作逻辑。上下文越有约束，概率分布通常越集中。

实验 B：只改一个词，观察分布变化

比较：

“银行门口停着一辆 ____。”

“河岸边停着一辆 ____。”

“赛车场上停着一辆 ____。”

“银行”一词有金融机构和河岸两种含义。后续上下文会重新塑造你对前面词义和后面词语的概率判断。现代 LLM 的核心优势之一，就是能利用较长上下文动态改变表示。

实验 C：错误如何传播

假设问题是：“法国首都是哪里？它位于哪个国家？”如果第一步错误地产生“伦敦”，第二步模型接着自己的错误上下文，可能自然续写“它位于英国”。第二句在局部语法和语义上完全连贯，但整

个答案已经沿着错误轨迹继续。

科研连接 把这个现象记住

单步错误进入上下文后，会影响后续概率。这就是为什么长链推理、Agent 任务、多 Agent 通信需要验证、纠错和恢复机制。



本讲术语表

先会认，再在后续课程里深入

术语	零基础解释
LLM	Large Language Model，大语言模型。对大规模语言数据进行训练、能够根据上下文建模和生成文本/代码等的模型。
Language Model	语言模型。描述语言序列概率规律的模型。
Token	模型处理文本时使用的离散单位。可能是字、词、词的一部分或符号组合。第2讲重点。
Parameter	参数。模型内部通过训练得到的可调数值。
Training	训练。利用数据与优化算法改变模型参数的过程。
Inference	推理/生成。用训练好的参数处理新输入并产生输出。
Context	上下文。模型当前可见、用于条件化预测的信息。
Next-token Prediction	下一个 token 预测。根据已有上下文估计下一个 token 的概率。
Autoregressive	自回归。把已生成的结果继续作为输入，逐步生成后续 token。
Probability Distribution	概率分布。对多个候选 token 分配不同概率。
GPT	Generative Pre-trained Transformer，一类生成式预训练 Transformer 模型家族/路线。
Agent	智能体。通常指能围绕目标感知状态、选择行动、调用工具并利用反馈继续决策的系统。

14

练习与答案

先自己回答，再对照答案

练习题

1. 用一句话解释“语言模型”是什么。
2. 为什么说 LLM 的最基础生成过程是“反复预测下一个 token”？
3. 模型输出概率最高的 token，是否意味着它一定是真实事实？为什么？
4. “模型参数”和“当前对话上下文”有什么区别？
5. 普通聊天时，模型是否每回答一次都重新训练参数？
6. ChatGPT 和 GPT 模型是完全同一个概念吗？
7. 为什么“预测下一个 token”这种简单训练目标有可能产生翻译、编程等复杂能力？
8. 为什么长任务里一个早期错误可能影响后续很多步骤？
9. Agent 与普通 LLM 相比，通常多了哪些系统组件？
10. 如果一个模型回答得非常流畅、自信，你能否据此判断内容一定正确？

参考答案

1. 语言模型是根据已有上下文，对后续语言序列建立概率并进行预测/生成的模型。
2. 因为每产生一个 token 后，模型把它接回上下文，再计算下一步分布，如此循环直到停止。
3. 不一定。概率反映的是在当前上下文和模型学到的统计规律下“多合适”，不是独立的事实核验。
4. 参数是训练写入模型内部的长期数值结构；上下文是当前推理时临时提供给模型的信息。
5. 通常不会。普通聊天主要属于 inference；参数训练是另一个昂贵过程。
6. 不是。GPT 是模型/模型家族，ChatGPT 是围绕模型构建的产品与系统。

7. 因为要在大量复杂上下文中持续预测准确，模型必须捕捉语法、语义、事实、代码结构和部分可泛化计算模式。
8. 因为后续生成以已经产生的内容为条件，早期错误会进入后续上下文，改变之后的概率。
9. 通常包括目标、状态、规划/决策、工具调用、记忆、行动执行、环境反馈和控制逻辑等。
10. 不能。表达风格不是事实证据；需要外部数据、检索、工具或独立验证。



一页总结与下一讲预告

把第一套心智模型固定下来

本讲最重要的 8 句话

核心概念	1. LLM 首先是一种语言概率模型，不是魔法盒。
核心概念	2. 它的经典核心任务是：根据上下文预测下一个 token。
核心概念	3. 每一步输出的本质是概率分布，最终生成是连续多步选择的结果。
核心概念	4. 简单的训练目标可以迫使模型形成复杂的内部表示与能力。
核心概念	5. “语言合理”不等于“事实真实”，这解释了许多幻觉现象。
核心概念	6. Training 改参数；Inference 主要用参数。普通对话通常不是重新训练。
核心概念	7. LLM 是模型层，聊天产品和 Agent 是建立在模型之上的系统层。
核心概念	

8. Agent Systems

的很多核心问题，来自概率模型在长流程、工具、记忆和多智能体交互中的误差传播与控制。

下一讲：Token——模型眼里的文字到底是什么？

下一讲会真正进入 LLM 的“输入端”：为什么模型不能直接看到文字？一句中文到底会被切成几个 token？Token ID 是什么？为什么 API 按 token 计费？为什么上下文 1M 指的不是 100 万个汉字？

例子 预习只需要想一个问题

“人工智能”这四个汉字，在模型眼里一定是四个 token 吗？答案：不一定。下一讲解释为什么。

建议延伸阅读（可选，不要求现在读）

Vaswani et al., 2017, Attention Is All You Need — Transformer 奠基论文。

Brown et al., 2020, Language Models are Few-Shot Learners —
大规模自回归语言模型能力的重要代表工作。

Ouyang et al., 2022, Training language models to follow instructions with human
feedback — 指令遵循与 RLHF 的代表工作。

本讲到这里结束。不要急着背术语；先把“上下文 → 概率分布 → 选择 token →
接回上下文 → 再预测”这条主线真正理解。